Image classification with Hybrid Quantum Convolutional Neural Networks

[Q]CML2

1st Miguel Castela
Dei, Universidade de Coimbra
uc2022212972@student.uc.pt

2nd Professor Jorge Lobo
Deec, Universidade de Coimbra
jlobo@isr.uc.pt

TABLE OF CONTENTS

Abstract 1

Introduction and methodology 1

First evaluations with the Qiskit tutorials 1
First tutorial 1
Second tutorial 4

Diferent types of QNN 6

Statistics 7
First tutorial 7
Second tutorial 7

What works best and creating a custom HCQNN 8
Trying to manually process the images using NEQR and FRQI 8
Deciding between Adam and COBYLA . 9
Deciding between z and COBYLA 9
Deciding the ansatz 9
The output layer 9
adding evolutionary operators 9
Pooling and convolution layers 10
Creating the custom HCQNN 10
tests 11
second dataset 11
MNIST dataset 12
results of the three HQNN with a more chanlenging dataset 12

Running custom HCQNN on real hardware 13

Conclusion 13

References 13

References 13

ABSTRACT

Abstract

INTRODUCTION AND METHODOLOGY

Artificial neural networks are currently one of the most popular models used in machine learning. A subcategory of these called convolutional neural networks (CNN) is par- ticularly popular. These are networks mainly used for the task of image recognition. Using pooling and convolution layers they extract structured information about images. THis project will be related to CNN topic but also to Quantum computing. Quantum machine learning leverages the principles of quantum mechanics to process information, allowing for the creation of algorithms that can potentially outperform classical counterparts. By harnessing quantum bits or qubits, these algorithms can explore multiple possibilities simultaneously, offering a unique approach to solving complex problems. While still in the early stages of development, quantum machine learning holds promise for revolutionizing certain computational tasks, particularly those involving large-scale optimization and data analysis.

FIRST EVALUATIONS WITH THE QISKIT TUTORIALS

First tutorial

The first step of this project was using the Qiskit framework, combined with pythorch to construct a HQNN. After studying the first three qiski tutorials (”Quantum Neural Networks”, ”Neural Network Classifier” & ”Training a Quantum Model on a Real Dataset”) i began to implement the first version of the QNN. This was done by implementing the ”Torch Connector and Hybrind QNNs” tutorialm:

- The first step is to initialize the Estimator. The Estimator is a class in Qiskit that allows you to estimate the expectation values of quantum circuits. It is used to evaluate the performance of quantum models by measuring their outputs.

```
estimator = Estimator()
```

- I then, using the torchvision API [1], load the MNIST training dataset(The MNIST database is a large database of handwritten digits [2]) and apply a transformation to convert images to tensors.
  A tensor is a generalization of vectors and matrices to any number of dimensions. They are the fundamental data structure in PyTorch, similar to NumPy arrays but with additional capabilities for GPU acceleration.

```
X_train = datasets.MNIST(
    root="./data", train=True,
    download=True,
    transform=transforms.Compose(
        [transforms.ToTensor()])
)
```

- I filter the dataset to keep only digits labeled 0 and 1 (first `n_samples` of each)
  I use the `np.append` function to concatenate the indices of the two classes (0 and 1) and then filter the dataset using these indices. The `np.where` function returns the indices of elements in an array that satisfy a given condition.
  The `X_train.data` and `X_train.targets` attributes are then updated to include only the selected samples.
  The `X_train.data` attribute contains the image data, while `X_train.targets` contains the corresponding labels.

```
idx = np.append(
    np.where(
        X_train.targets == 0)
        [0][:n_samples]
    , np.where(
        X_train.targets == 1)
        [0][:n_samples]
)
X_train.data = X_train.data[idx]
X_train.targets = X_train.targets[idx]
```

- I define the PyTorch DataLoader [3] for the filtered training data, this is used to load the data in batches during training. The `shuffle=True` argument ensures that the data is shuffled before each epoch, which helps improve the model's generalization.

```
train_loader = DataLoader(X_train,
batch_size=batch_size, shuffle=True)
```

- I load the MNIST dataset again to create the test set, this time with `train=False` to indicate that we want the test set. The same transformation is applied as before.
  I again filter the test dataset to keep only digits labeled 0 and 1 (first `n_samples` of each) and define the DataLoader for the filtered test data.

```
n_samples = 50
X_test = datasets.MNIST(
    root="./data",
    train=False, download=True,
    transform=transforms.Compose(
        [transforms.ToTensor()])
)
...
```

- Then I create the quantum neural network. At this point, a key decision must be made: **how to encode the classical input features into a quantum state** — a process known as *feature mapping* or *data encoding*.
  Qiskit provides several options for this, the most common being:
  - **ZFeatureMap [4]**
    A simple quantum feature map where each input value is encoded as a rotation on a single qubit using Rz gates. This approach does **not include entanglement** and is typically used for simpler datasets or models with lower computational complexity.
  - **ZZFeatureMap [5]**
    A more expressive feature map that includes both single-qubit rotations and **entangling gates (ZZ interactions)** between qubits. This allows the model to capture complex relationships between features, making it more suitable for richer or more structured datasets.

  These feature maps are not just about storing the features — they are essential for **embedding classical data (such as pixel values from images)** into the quantum circuit so that the quantum neural network can process it.
  Other alternatives to these include:
  - **PauliFeatureMap [6]** – a generalized feature map that supports different combinations of Pauli operators.
  - **Custom feature maps** – where users design their own encoding strategies tailored to specific datasets or problems. This was explored in the NEQR and FRQI section

  The choice of feature map directly influences the expressiveness and performance of the quantum model. In this case, i used the `ZZFeatureMap`.
  The next choice is the **ansatz** (the quantum circuit structure). The ansatz defines the architecture of the quantum neural network, including the number of qubits, layers, and gates used. Qiskit provides several predefined ansatz classes, such as:
  - **EfficientSU2 [7]**
    A popular ansatz that uses a combination of single-qubit rotations and entangling gates. It is designed to be efficient in terms of circuit depth and is suitable for a wide range of problems.
  - **RealAmplitudes [8]**
    This ansatz uses real-valued parameters and is often used for variational algorithms. It can be more expressive than EfficientSU2 but may require more qubits and gates.

– **TwoLocal [9]**
A flexible ansatz that allows users to specify the types of gates and the connectivity between qubits. It can be tailored to specific problems but may require more tuning.

The choice of ansatz is crucial as it determines the expressiveness and complexity of the quantum model. In this case i used the RealAmplitudes.

```
def create_qnn():
    feature_map = ZZFeatureMap(2)
    ansatz = RealAmplitudes(
        2, reps=1)
    qc = QuantumCircuit(2)
    qc.compose(
        feature_map, inplace=True)
    qc.compose(
        ansatz, inplace=True)

    qnn = EstimatorQNN(
        circuit=qc,
        input_params=
        feature_map.parameters,
        weight_params=
        ansatz.parameters,
        input_gradients=True,
        estimator=estimator,
    )
    return qnn

qnn4 = create_qnn()
```

- The next step is to define the neural network that includes a quantum layer.

  This class Net [10] is a custom neural network built using PyTorch. It combines classical layers (such as convolutional and linear layers) with a quantum neural network (QNN) layer created earlier with Qiskit. The QNN is integrated using the TorchConnector, which allows it to be used just like any other PyTorch layer and supports hybrid quantum-classical training via backpropagation.

  There are two convolutional layers, typically used for image data.
  $Conv2d(1, 2, kernel_size = 5)$ means the first layer takes 1 input channel $(grayscaleimage)$ and outputs 2 feature maps using 5x5 filters.
  The second layer takes 2 channels and produces 16 feature maps.

  Dropout2d [11] is a regularization layer that randomly zeros out entire channels during training to prevent overfitting.

  fc1 and fc2 are fully connected (linear) layers. The first one takes the flattened output of the convolutional layers (256 features) and reduces it to 64 features. The second one takes the 64 features and outputs 2 features, which are then passed to the QNN.

  The Qiskit quantum neural network is then wrapped using the TorchConnector, allowing it to be used as a layer in the PyTorch model. The QNN takes the 2-dimensional input from fc2 and produces a 1-dimensional output.

  The parameters (weights) of the quantum circuit are initialized randomly in the interval $[-1, 1]$.
  Finally, fc3 is another fully connected layer that takes the 1-dimensional output from the QNN and produces a final output of 2 features (which can be interpreted as probabilities for binary classification).

```
class Net(Module):
def __init__(self, qnn):
    super().__init__()
    self.conv1 =
    Conv2d(1, 2, kernel_size=5)
    self.conv2 =
    Conv2d(2, 16, kernel_size=5)
    self.dropout =
    Dropout2d()
    self.fc1 =
    Linear(256, 64)
    self.fc2 =
    Linear(64, 2)
    self.qnn
    = TorchConnector(qnn)
    self.fc3 = Linear(1, 1)
```

- The next step is to do the Forward pass.
  It applies the first convolutional layer with a ReLU [12] activation, followed by 2x2 max pooling (downsampling). The same steps are repeated for the second convolutional layer.
  After feature extraction with convolutional layers and dropout for regularization, the data is flattened and passed through two fully connected layers (fc1 and fc2). The output, a 2D vector, is then processed by the quantum neural network (QNN) layer. Its result goes through another linear layer (fc3), and the final output is a concatenation of the prediction and its complement, representing probabilities for binary classification (digit 0 vs 1).

```
def forward(self, x):
    x = F.relu(self.conv1(x))
    x = F.max_pool2d(x, 2)
    x = F.relu(self.conv2(x))
    x = F.max_pool2d(x, 2)
    x = self.dropout(x)
    x = x.view(x.shape[0], -1)
    x = F.relu(self.fc1(x))
    x = self.fc2(x)
    x = self.qnn(x)  # apply QNN
    x = self.fc3(x)
    return cat((x, 1 - x), -1)
```

- Finally i need to inicialize the mode, optimizer and loss function.
  The optimizer is responsible for updating the model's parameters during training. In this case,

the Adam optimizer [13] is used with a learning rate of 0.01.
The loss function is used to measure the difference between the predicted output and the true labels. In this case, the Negative Log Likelihood Loss is used.

Adam is a first-order gradient-based optimizer that combines momentum, which uses past gradients, and adaptive learning rates, which scale the step size based on how frequently a parameter is updated. It is commonly used for most deep learning tasks due to its speed and robustness. One of its key advantages is the adaptive learning rate per parameter, which often leads to good convergence. However, one downside of Adam is that it can sometimes generalize worse than Stochastic Gradient Descent (SGD).
The model is trained for 10 epochs. In each epoch, it iterates over the training batches:

- Gradients are reset with `optimizer.zero_grad()`.
- A forward pass is performed to get the model's predictions.
- The loss between predictions and true labels is computed.
- A backward pass computes gradients.
- The model updates its weights using the optimizer.
- Loss values for the epoch are stored and averaged.

- A new quantum neural network is created and the trained weights are loaded:

The model is then evaluated on the test dataset. First, it is set to evaluation mode using `model.eval()`. Gradient computation is disabled using `no_grad()` to speed up inference and save memory.

For each batch in the test set:

- The model outputs a prediction for the input data.
- If the output has an unexpected shape, it is reshaped appropriately.
- The predicted label is obtained using `argmax`.
- The number of correct predictions is counted by comparing with the true labels.
- The loss is computed using the same loss function as in training.

At the end, the average loss and accuracy are printed

Second tutorial

The second tutorial added the concepts of pooling layers, creating the pooling and convolution circuits and a custom dataset.

- The first step is to define a two bit convolutional circuit. This circuit is used to perform a convolution operation on two qubits, which is a fundamental operation in quantum convolutional neural networks (QCNNs).

```
def conv_circuit(params):
    target = QuantumCircuit(2)
    target.rz(-np.pi / 2, 1)
    target.cx(1, 0)
    target.rz(params[0], 0)
    target.ry(params[1], 1)
    target.cx(0, 1)
    target.ry(params[2], 1)
    target.cx(1, 0)
    target.rz(np.pi / 2, 0)
    return target
params = ParameterVector("\theta", length=3)
circuit = conv_circuit(params)
```

- Convolutional Layer A convolutional layer is constructed by composing multiple convolutional circuits. The result is a quantum circuit that can be applied to the qubits.

```
def conv_layer(num_qubits, param_prefix):
    qc = QuantumCircuit(
        num_qubits, name="Convolutional Layer")
    qubits = list(
        range(num_qubits))
    param_index = 0
    params = ParameterVector(
        param_prefix
        , length=num_qubits * 3)
    for q1, q2 in zip(
        qubits[0::2], qubits[1::2]):
        qc = qc.compose(
            conv_circuit(
                params[
                    param_index : (
                        param_index + 3)])
                , [q1, q2])
        qc.barrier()
        param_index += 3
    for q1, q2 in zip(
        qubits[1::2], qubits[2::2] + [0]):
        qc = qc.compose(
            conv_circuit(
                params[
                    param_index : (
                        param_index + 3)])
                , [q1, q2])
        qc.barrier()
        param_index += 3

    qc_inst = qc.to_instruction()

    qc = QuantumCircuit(num_qubits)
    qc.append(qc_inst, qubits)
    return qc
```

- A pooling circuit is defined. This circuit is used to perform a pooling operation on two qubits, which is another fundamental operation in quantum convolutional neural networks (QCNNs). The pooling operation reduces the dimensionality of the data while retaining important features.

```
def pool_circuit(params):
```

```
    target = QuantumCircuit(2)
    target.rz(-np.pi / 2, 1)
    target.cx(1, 0)
    target.rz(params[0], 0)
    target.ry(params[1], 1)
    target.cx(0, 1)
    target.ry(params[2], 1)
    return target
```

- The pooling layer is created by composing several pooling circuits.

```
def pool_layer(sources
, sinks, param_prefix):
    num_qubits = len(sources)
    + len(sinks)
    qc = QuantumCircuit(num_qubits
    , name="Pooling Layer")
    param_index = 0
    params = ParameterVector(
        param_prefix
        , length=num_qubits // 2 * 3)
    for source
    , sink in zip(sources, sinks):
        qc = qc.compose(
            pool_circuit(
                params[param_index : (
                    param_index + 3)])
                , [source, sink])
        qc.barrier()
        param_index += 3

    qc_inst = qc.to_instruction()

    qc = QuantumCircuit(num_qubits)
    qc.append(qc_inst
    , range(num_qubits))
    return qc
```

- Dataset Generation A synthetic dataset of images is created to train the QCNN. Each image is either a horizontal or vertical line. The images are generated using the `generate_dataset` function.
- Model Training and Evaluation The dataset is split into training and testing sets. The classifier is then trained using a quantum neural network with the defined convolutional and pooling layers.

```
images, labels = generate_dataset(10)


train_images, test_images
, train_labels
, test_labels = train_test_split(
    images
, labels, test_size=0.3
, random_state=246
)
```

- Define the callback function: The callback function is used to track the objective function value during training. It updates the plot of the objective function value against the iteration number.

```
def callback_graph(
weights, obj_func_eval):
    clear_output(wait=True)
    objective_func_vals.append(
        obj_func_eval)
```

- Define the ansatz and feature map:
  We start by initializing a feature map (ZFeatureMap) and a quantum circuit (ansatz) that will serve as our ansatz. This circuit is responsible for transforming input data into a quantum state.

```
feature_map = ZFeatureMap(8)
feature_map = ZFeatureMap(8)
ansatz = QuantumCircuit(8, name="Ansatz")
```

- Define the pooling and convolutional layers:

```
ansatz.compose(
    conv_layer(8, "c1")
    , list(range(8))
    , inplace=True)

ansatz.compose(
    pool_layer([0, 1, 2, 3]
    , [4, 5, 6, 7], "p1")
    , list(range(8))
    , inplace=True)

ansatz.compose(
    conv_layer(4, "c2")
    , list(range(4, 8))
    , inplace=True)

ansatz.compose(
    pool_layer([0, 1], [2, 3], "p2")
    , list(range(4, 8))
    , inplace=True)

ansatz.compose(
    conv_layer(2, "c3")
    , list(range(6, 8))
    , inplace=True)

# Third Pooling Layer
ansatz.compose(pool_layer([0], [1], "p3")
, list(range(6, 8))
, inplace=True)
```

  The convolutional and pooling layers are defined using the conv_layer and pool_layer functions. These layers are composed into the ansatz circuit, which will be used in the quantum neural network. The convolutional layers are applied first, followed by the pooling layers. The pooling layers reduce the dimensionality of the data while retaining important features. The final ansatz circuit is a combination of these layers.
- Then, the ansatz circuit is combined with the feature map. The observable is defined as a SparsePauliOp [14] with the operator (”Z” + ”I” * 7), which will measure the Z component of the state on the first qubit while leaving the others unchanged. This observable is used to extract information from the quantum state after the ansatz has been applied. The circuit is decomposed to avoid data duplication when passed to the quantum neural network (QNN). This ensures the circuit is executed efficiently in the EstimatorQNN [15] model.

```
circuit = QuantumCircuit(8)
circuit.compose(feature_map
, range(8), inplace=True)
```

```
circuit.compose(ansatz
, range(8), inplace=True)

observable =
SparsePauliOp.from_list([("Z" + "I" * 7, 1)])

qnn = EstimatorQNN(
    circuit=circuit.decompose(),
    observables=observable,
    input_params=feature_map.parameters,
    weight_params=ansatz.parameters,
    estimator=estimator,
)
```

- The classifier is then initialized and trained using the training data. The NeuralNetworkClassifier is initialized with the QNN and the COBYLA optimizer (with a maximum of 200 iterations). A callback function (callback_graph) is passed to track the objective function during training. The classifier is then trained using the training data (x,y).
  During classifier.fit(), the following occurs:
  The NeuralNetworkClassifier performs a forward pass by computing the output for each input image x in `train_images`. The data is passed through the quantum circuit defined by `feature_map` and `ansatz`, and the output is compared to the true labels y. The quantum neural network (qnn) is a parameterized model, and this forward pass generates predictions by applying the quantum operations defined in the ansatz to the data encoded in the quantum feature map.
  Backward Pass:
  The backward pass involves calculating the gradients of the loss function with respect to the parameters (weights) of the network and updating those parameters to minimize the loss. This is where the optimization process happens. In this case, the optimizer is COBYLA [16], which is a gradient-free optimization algorithm. While COBYLA does not explicitly require gradients (as it doesn't rely on gradient-based updates), it still operates by iterating through the optimization process to find the best set of weights for the QNN by minimizing the objective function.
  Differenced betweenn Adam and COBYLA:
  - **Adam Optimizer**:
    - **Momentum**: Tracks the exponentially decaying average of past gradients to help avoid oscillations and speed up convergence.
    - **Adaptive Learning Rates**: Adjusts the learning rates for each parameter based on past gradients, improving convergence rates for different parameters in neural networks.
    - **Bias Correction**: Corrects the initial bias in the moment estimates, especially for small t (iteration steps).
  - **Key Hyperparameters of Adam**:
    - `alpha`: Learning rate.
    - `beta1`: Exponential decay rate for the first moment estimate.
    - `beta2`: Exponential decay rate for the second moment estimate.
    - `epsilon`: Small constant to prevent division by zero.
  - **COBYLA Optimizer**:
    - **Derivative-Free**: COBYLA is a derivative-free optimization method suitable for problems without gradients or where gradient calculation is expensive.
    - **Key Difference from Adam**: Unlike Adam, COBYLA does not rely on gradients, making it suitable for non-differentiable problems.
  - **Alternative to `optimizer.zero_grad(set_to_none=True)`**:
    - COBYLA does not require clearing gradients since it does not use gradients.
    - For gradient-based optimizers (like Adam), you typically use `optimizer.zero_grad()`, but COBYLA does not need such a step.
    - When using COBYLA in frameworks like PyTorch, avoid gradient-based methods entirely and focus on function evaluations.

```
initial_point =
 np.random.random(qnn.num_weights)
classifier = NeuralNetworkClassifier(
    qnn,
    optimizer=COBYLA(maxiter=200),
    callback=callback_graph,
    initial_point=initial_point,
)

x = np.asarray(train_images)
y = np.asarray(train_labels)

objective_func_vals = []
plt.rcParams["figure.figsize"] = (12, 6)

classifier.fit(x, y)

y_predict = classifier.predict(test_images)

x = np.asarray(test_images)
y = np.asarray(test_labels)
```

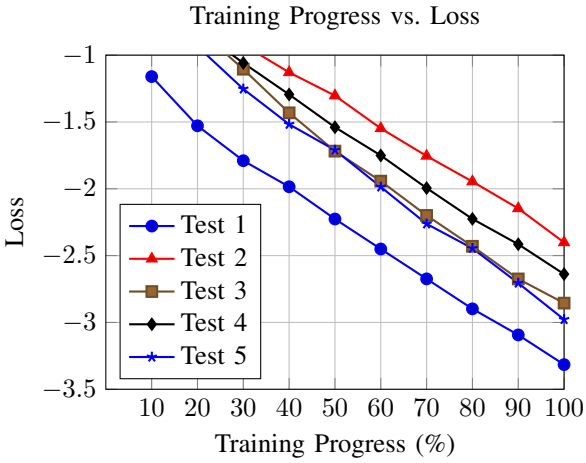
DIFERENT TYPES OF QNN

The first tutorial focuses on a direct application of a quantum neural network (QNN) in a hybrid classical-quantum neural network, using a known dataset like MNIST. The model begins with classical processing (convolutions, pooling, fully connected layers), passes through quantum circuits for feature extraction, and concludes with a classical layer for classification.

In contrast, the second tutorial adopts a more experimental approach, where data is processed classically and then converted into a quantum format. The QCNN uses convolutional and pooling layers to extract features, and the NeuralNetworkClassifier trains the model. Precision is evaluated, and results are displayed in a classical manner.

STATISTICS

First tutorial



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-3.4312	-2.4912	-2.9313	-2.6458	-3.1129
accuracy	100%	99%	100%	100%	100%

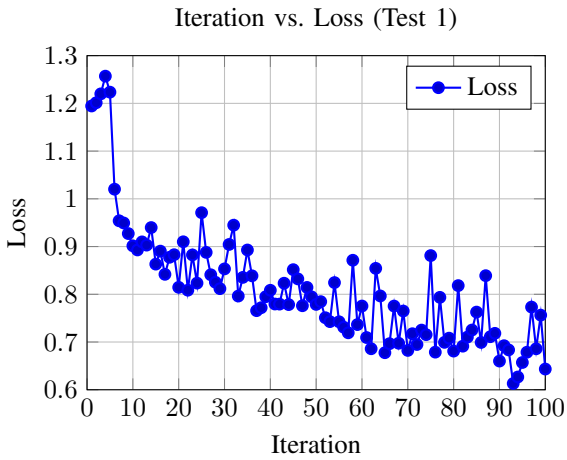
TABLE I: accuracy and final loss of the tests

These tests were simulated locally, with a manual seed differing for each test. The accuracy was based on 50 training samples and 50 test samples, with the model trained for 10 epochs (One full pass through the entire training dataset). The final loss was calculated after the training process.

The results show that the model achieved high accuracy across all tests, with the final loss values indicating good convergence. The training progress was visualized, showing a steady decrease in loss as the training progressed.

While all runs achieved high accuracy ($\geq$99%), the final loss varied, with a mean of -2.92 and a standard deviation of 0.34. This indicates that the model is robust and generalizes well, though training outcomes still depend moderately on initialization.

Second tutorial



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-3.4312	-2.4912	-2.9313	-2.6458	-3.1129
accuracy	66.67%	66.67%	100%	100%	33.33%

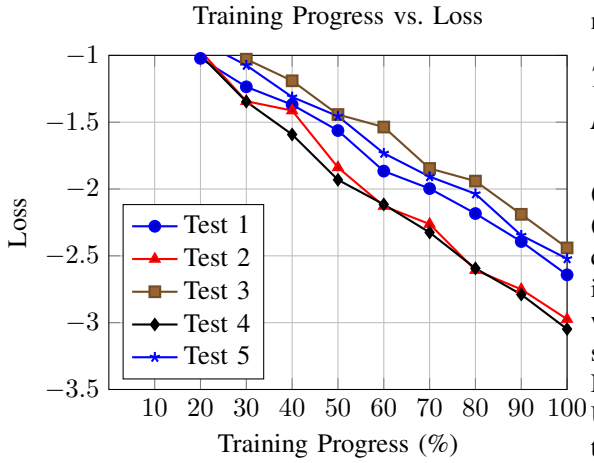
TABLE II: accuracy and final loss of the tests

The model was trained us ing the COBYLA optimizer for a maximum of 200 iterations. The results show that the model achieved varying accuracy across different tests, with some tests achieving 100% accuracy while others had lower performance. The final loss values indicate that the model converged to different levels of performance depending on the test. The training progress of Test 1 was visualized, showing a steady decrease in loss as the iterations progressed. The loss values fluctuated but generally trended downwards, indicating that the model was learning and improving its performance over time.

The performance disparity between the two tutorials stems from several architectural and implementation differences. The first tutorial integrates the quantum model as a hybrid layer inside a classical convolutional neural network (CNN), which first extracts spatial features from the MNIST images using two Conv2d layers followed by fully connected layers, before embedding a shallow 2-qubit quantum neural network (QNN) via TorchConnector. Conversely, the second tutorial adopts a fully quantum-centric approach with quantum convolutional and pooling circuit architecture that attempts to encode artifical images generated. However, it lacks any form of classical preprocessing or feature extraction, making it harder for the quantum model to generalize from high-dimensional input data. The usage of zfeature map (although with more qubits) in the second tutorial also limits the model's ability to capture complex patterns compared to the first tutorial's classical CNN layers. Additionally, the second tutorial's reliance on a derivative-free optimizer (COBYLA) contrasts with the first tutorial's Adam optimizer, which is more suited for gradient-based learning in hybrid models. These factors contribute to

the observed differences in performance and accuracy between the two tutorials.

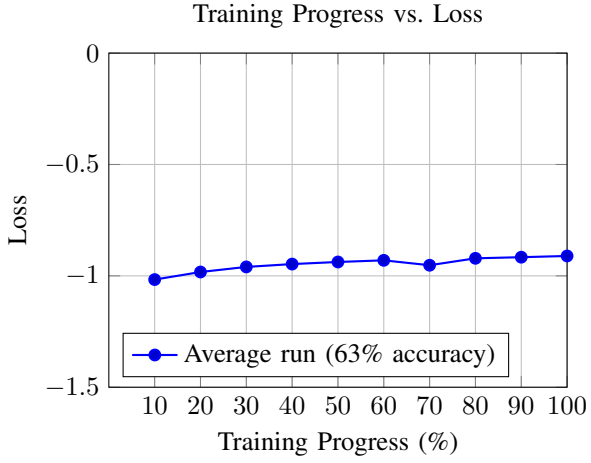
The first tutorial also found success with the dataset used in the second tutorial.



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-2.5541	-3.2172	-2.7470	-3.1065	-2.7612
accuracy	99%	100%	100%	100%	100%

TABLE III: accuracy and final loss of the tests

while the second tutorial, when trying to classify the dataset from the first one, achived around 50% accuracy with very little loss convergence. This was due to the way the second tutorial treats the data in a more quantum-centric manner, the images need to be resized to fit the quantum model, losing its features and becoming too noisy.



WHAT WORKS BEST AND CREATING A CUSTOM HCQNN

Based on the results obtained from the first two tutorials, i developed a custom hybrid quantum convolutional neural network (HCQNN) that combines the best features of both approaches, while also testing classical image processing techniques.

Like in the first tutorial, i load the Estimator() and MNIST dataset. After defining the seed, batch size and number of samples, After loading the MNIST training set, i apply transformation to convert images to tensors, like in the first tutorial. I then load 50 samples for each of digits 0 and 1 from the test set. I then create the QNN, with a function that lets me define the encoding methods(NEQR, FRQI or ZZFeatureMap).

Trying to manually process the images using NEQR and FRQI

Flexible Representation of Quantum Images (FRQI): This method encodes both the color (grayscale intensity) and position of each pixel into a quantum state. Specifically, the intensity information is represented by rotating a qubit to a specific angle, while the pixel positions are mapped using basis states.

Novel Enhanced Quantum Representation (NEQR): Unlike FRQI, NEQR uses multiple qubits to represent the grayscale value of each pixel directly, allowing for a more precise encoding of image information.

I tried to implement the NEQR and FRQI methods in my implementation, instead of using ZZFeatureMap. These were the functions used:

```
def frqi_encoding(image):
    n = int(np.log2(len(image)))
    qreg = QuantumRegister(n + 1)
    qc = QuantumCircuit(qreg)

    for i in range(n):
        qc.h(qreg[i])

    for i, pixel in enumerate(image):
        theta = 2 * np.arccos(
            np.sqrt(pixel))
        bin_index = format(
            i, f'0{n}b')
        for j, bit in enumerate(
            bin_index):
            if bit == '1':
                qc.x(qreg[j])
        qc.cry(theta, qreg[n-1]
        , qreg[n])
        for j, bit in enumerate(
            bin_index):
            if bit == '1':
                qc.x(qreg[j])
    qc.custom_num_qubits = n + 1
    return qc

def neqr_encoding(image):
n = int(np.log2(len(image)))
qreg = QuantumRegister(n + 8)
qc = QuantumCircuit(qreg)

for i in range(n):
    qc.h(qreg[i])

for i, pixel in enumerate(image):
    bin_pixel = format(
        int(pixel * 255), '08b')
    bin_index = format(i
    , f'0{n}b')

    for j, bit in enumerate(
        bin_index):
        if bit == '1':
```

```
                qc.x(qreg[j])

        for j, bit in enumerate(
            bin_pixel):
            if bit == '1':
                qc.mcx(qreg[:n]
                , qreg[n + j])

        for j, bit in enumerate(
            bin_index):
            if bit == '1':
                qc.x(qreg[j])

    return qc
```

Deciding between Adam and COBYLA

In this hybrid quantum-classical neural network setup, using the Adam optimizer is more appropriate than COBYLA. The model integrates both classical layers (such as convolutional and fully connected layers) and a quantum neural network (QNN) wrapped with TorchConnector, which enables automatic differentiation through PyTorch's autograd system. This setup relies on gradient-based optimization, where the loss.backward() call computes gradients, and optimizer.step() updates the parameters accordingly. In contrast, COBYLA is a gradient-free optimizer from SciPy that does not integrate with PyTorch's autograd and cannot update parameters through backpropagation. Using COBYLA in this context would prevent the model from training properly, as it cannot optimize the classical network components or interface with PyTorch's training loop. Therefore, for training hybrid models end-to-end within PyTorch, gradient-based optimizers like Adam should be used to ensure all components are updated correctly.

Deciding between zfeaturemap and zzfeaturemap

The results of using NEQR and FRQI instead of quantum feature maps were not as good as expected. The NEQR and FRQI methods are more complex and require careful tuning of parameters, which can lead to longer training times and less intuitive results.

Because of this, the decision came down to using the ZFeatureMap or ZZFeatureMap.
The ZFeatureMap is a simpler feature map that encodes classical data into quantum states using single-qubit rotations. It is computationally less expensive and easier to implement, making it suitable for smaller datasets or simpler models. However, it may not capture complex relationships between features as effectively as the ZZFeatureMap. The ZZFeatureMap, on the other hand, includes both single-qubit rotations and entangling gates (ZZ interactions) between qubits. This allows the model to capture complex relationships between features, making it more suitable for richer or more structured datasets. However, it is computationally more expensive and may require more tuning to achieve optimal performance.

Deciding the ansatz

The ansatz is a crucial component of quantum neural networks, as it defines the structure and complexity of the quantum circuit used for feature extraction. The choice of ansatz can significantly impact the performance of the model.
The chosen ansatz, from the ones listed in the first tutorial, was the TwoLocal ansatz. This ansatz is flexible and allows users to specify the types of gates and the connectivity between qubits. It can be tailored to specific problems, making it suitable for a wide range of applications. However, it may require more tuning compared to other ansatzes like EfficientSU2 or RealAmplitudes.
The parameters chosen were:$ansatz = TwoLocal(num_qubits = 2, rotation_blocks =' ry', entanglement_blocks =' cz', reps = 1)$.

The output layer

I used nn.sequential to create the output layer, which is a simple way to stack layers in PyTorch. The output layer consists of a linear layer followed by a softmax activation function. The linear layer maps the output of the quantum neural network (QNN) to the desired number of classes (in this case, 2 for binary classification). The softmax activation function converts the raw output scores into probabilities, making it suitable for multi-class classification tasks.
This is better that the previous approach, where the output was a concatenation of the prediction and its complement. The new approach is more straightforward and aligns better with standard practices in neural network design.

adding evolutionary operators

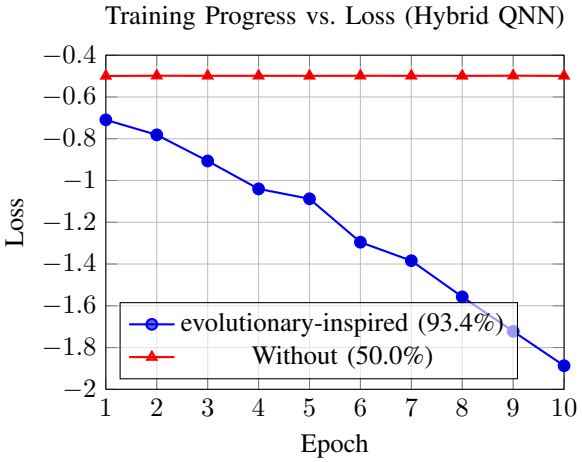
To complement the gradient-based optimization performed by the Adam optimizer, evolutionary-inspired strategies were also applied to the model parameters. This hybrid optimization approach introduces stochastic variation through crossover and mutation operators, inspired by genetic algorithms. These operators are applied at the end of each training epoch to promote exploration of the parameter space and potentially escape local minima.

The crossover method used is a single-point crossover, commonly applied in evolutionary algorithms to combine traits from two parent solutions. It works by first flattening both parameter tensors into one-dimensional vectors. A random crossover point is then selected somewhere between the first and last index. At this point, the vectors exchange their tail segments, producing two new offspring vectors that blend characteristics of both parents. These new vectors are subsequently reshaped back into the original tensor dimensions to maintain compatibility with the model. This process enables the model to explore new combinations of parameters, encouraging diversity in

the population while preserving useful traits from the previous generation.

The mutation method employed is a Gaussian mutation with per-element mutation probability, a standard approach for introducing variability into model parameters. This technique begins by generating a mask tensor of the same shape as the parameter tensor, where each element has a fixed probability (e.g., 5%) of being selected for mutation. For every element in the tensor, Gaussian noise with zero mean and a specified standard deviation (mutation strength, typically 0.1) is generated. This noise is then selectively added only to those elements flagged by the mask. As a result, a subset of the parameter values is perturbed by small, random amounts, introducing stochastic changes that help the model explore new solutions and avoid premature convergence.

Together, these evolutionary operators create a hybrid learning paradigm, blending backpropagation with biologically inspired stochastic search. While this adds computational overhead, it can improve convergence and robustness in non-convex and high-dimensional optimization settings, especially when combined with noisy quantum components.



These results had the same seed, batch size and 500 testing samples with 100 training. The model was trained for 10 epochs, with the final loss calculated after the training process.

The results show that the model achieved high accuracy with evolutionary operators, while the model without them did not improve significantly. The training progress was visualized, showing a steady decrease in loss as the training progressed.

The evolutionary operators significantly improved the model's performance, demonstrating the effectiveness of combining gradient-based optimization with stochastic search methods in hybrid quantum-classical neural networks.

Pooling and convolution layers

These were used like in the second tutorial, but only one of each was created.

Creating the custom HCQNN

the full code is as follows:

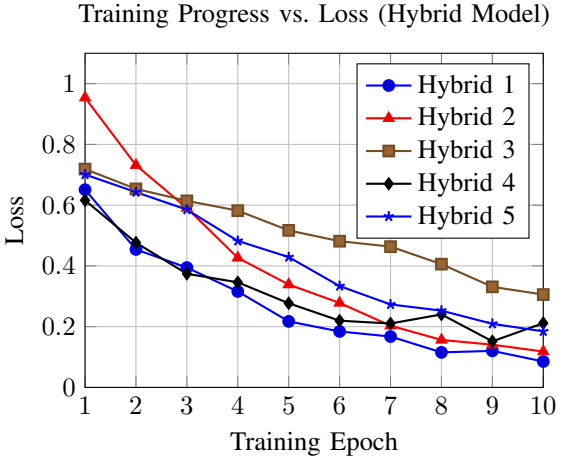
```
estimator = Estimator()


np.random.seed(121)

batch_size = 1
n_train_samples = 100
n_test_samples = 500

def load_filtered_fashion_mnist(
    train=True, n_samples=100):
    dataset = datasets.FashionMNIST(
        root="./data",
        train=train,
        download=True,
        transform=transforms.Compose(
            [transforms.ToTensor()])
    )

    selected_classes = [0, 2]
    idx = np.where(
        np.isin(
            dataset.targets,
            selected_classes))[0]

    idx0 = idx[
        dataset.targets[idx] == 0][
            :n_samples]
    idx2 = idx[
        dataset.targets[idx] == 2][
            :n_samples]
    idx = np.append(idx0, idx2)

    dataset.data = dataset.data[idx]
    dataset.targets = dataset.targets[idx]

    dataset.targets =
    torch.where(
        dataset.targets == 2,
        torch.tensor(1),
        dataset.targets)

    loader = DataLoader(dataset,
    batch_size=batch_size,
    shuffle=True)
    return loader

train_loader = load_filtered_fashion_mnist(
    train=True,
    n_samples=n_train_samples)
test_loader = load_filtered_fashion_mnist(
    train=False,
    n_samples=n_test_samples)

def create_qnn():
    feature_map = ZZFeatureMap(2)
    ansatz = TwoLocal(num_qubits=2,
    rotation_blocks='ry', entanglement_blocks='cz',
    reps=1)
    qc = QuantumCircuit(2)
    qc.compose(feature_map, inplace=True)
    qc.compose(ansatz, inplace=True)
    qnn = EstimatorQNN(
        circuit=qc,
        input_params=feature_map.parameters,
```

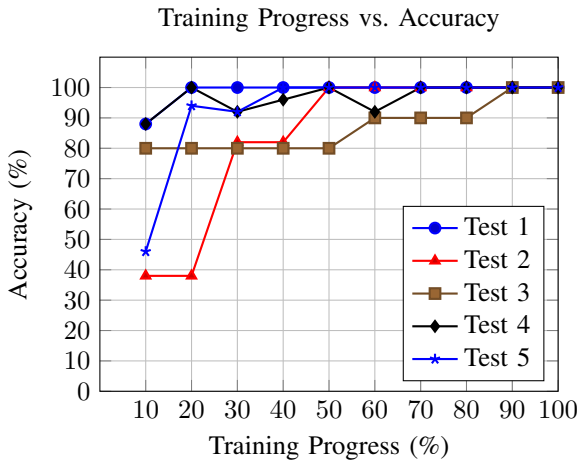
```
        weight_params=ansatz.parameters,
        input_gradients=True,
        estimator=estimator,
    )
    return qnn

qnn = create_qnn()

class HybridNet(Module):
    def __init__(self, qnn):
        super().__init__()
        self.conv1 = Conv2d(1, 4, kernel_size=5)
        self.pool = torch.nn.MaxPool2d(2)
        self.fc1 = Linear(4*12*12, 2)
        self.qnn = TorchConnector(qnn)
        self.fc2 = Linear(1, 1)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.pool(x)
        x = x.view(x.shape[0], -1)
        x = F.relu(self.fc1(x))
        x = self.qnn(x)
        x = self.fc2(x)
        # Output 2 classes as prob distribution
        return cat((x, 1 - x), dim=-1)

model = HybridNet(qnn)
optimizer = optim.Adam(model.parameters(),
lr=0.001)
loss_func = NLLLoss()

def crossover(param1, param2):
    flat1 = param1.flatten()
    flat2 = param2.flatten()
    length = flat1.size(0)
    point = torch.randint(
        1, length, (1,)).item()
    new1 = torch.cat([flat1[:point],
    flat2[point:]])
    new2 = torch.cat([flat2[:point],
    flat1[point:]])
    return new1.view(param1.shape),
    new2.view(param2.shape)

def mutation(param, mutation_rate=0.05,
 mutation_strength=0.1):
    mask = (torch.rand_like(param) <
    mutation_rate).float()
    noise = torch.randn_like(param) *
    mutation_strength
    param.data.add_(mask * noise)

def apply_evolution(model):
    params = list(model.parameters())
    n = len(params)
    for i in range(0, n - 1, 2):
        if params[i].shape ==
         params[i + 1].shape:
            new1, new2 = crossover(params[i],
             params[i + 1])
            params[i].data.copy_(new1)
            params[i + 1].data.copy_(new2)
        else:
            pass
    for i in range(n):
        mutation(params[i])

epochs = 10
loss_list = []

model.train()
for epoch in range(epochs):
    total_loss = []
    for data, target in train_loader:
        optimizer.zero_grad()
        output = model(data)
        loss = loss_func(output, target)
        loss.backward()
        optimizer.step()
        total_loss.append(loss.item())
    loss_list.append(sum(total_loss) /
     len(total_loss))
    print(f"Epoch {epoch+1}/{epochs} - Loss: {
        loss_list[-1]:.4f}")

    apply_evolution(model)



qnn_test = create_qnn()
model_test = HybridNet(qnn_test)
model_test.load_state_dict(
    torch.load(
        "model_evolution.pt"))
model_test.eval()

correct = 0
total_loss = []
with no_grad():
    for data, target in test_loader:
        output = model_test(data)
        if len(output.shape) == 1:
            output = output.unsqueeze(0)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).
        sum().item()
        loss = loss_func(output, target)
        total_loss.append(loss.item())
```

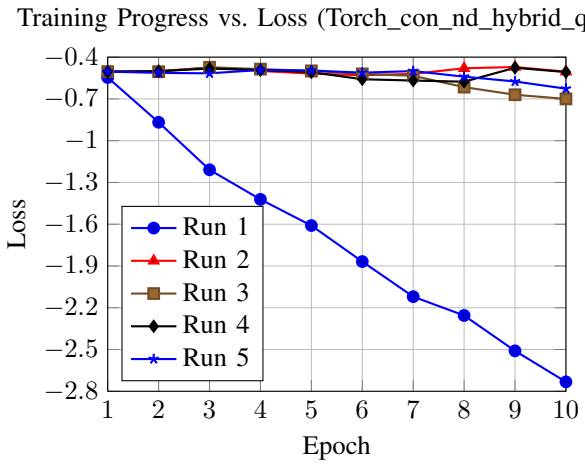
tests

second dataset

These were the results obtained with the artificial dataset used in the second tutorial, with the custom HCQNN:





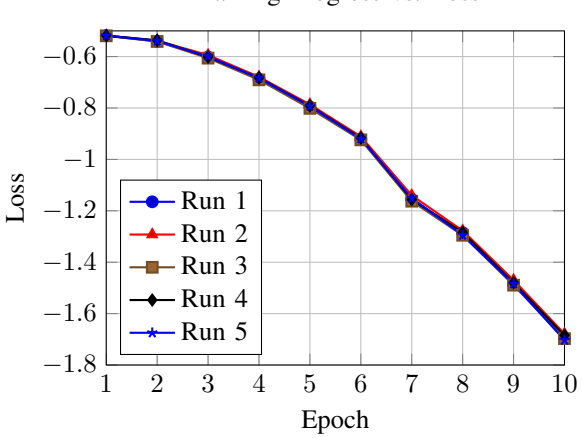


	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-2.6044	-0.4413	-0.8352	-0.4623	-0.7545
accuracy	93%	47%	89%	49%	89%

TABLE V: accuracy and final loss of the tests

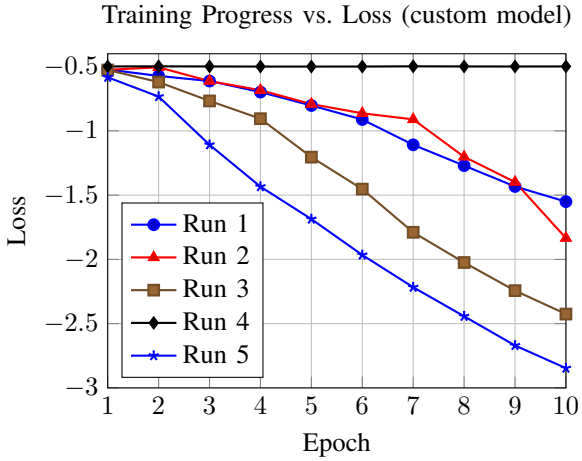
MNIST dataset

The results obtained with the MNIST dataset, using the same test and training size as in the first tutorial were as follows:



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-1.7407	-1.7141	-1.7451	1.7048	1.7465
accuracy	100%	99%	100%	100%	100%

TABLE IV: accuracy and final loss of the tests



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-1.3442	-1.7675	-2.2366	-0.4995	-2.6039
accuracy	91%	94%	93%	50%	93%

TABLE VI: accuracy and final loss of the tests

results of the three HQNN with a more chanlenging dataset

These tests were performed using the fashion MNIST dataset, which is more challenging than the MNIST dataset used in the first tutorial. The same traininf and testing size was used. I selected a subset of classes (0 and 2, ”T-shirt/top” and ”Pullover”) to keep binary classification.

The results obtained were as follows:

These results show that the custom HCQNN model performed well on the fashion MNIST dataset, achieving high accuracy across different tests. The final loss values indicate that the model converged to a good solution, with some tests achieving very low loss values. The training progress also shows a steady decrease in loss over the epochs, indicating that the model was learning effectively. The results, even with a small testing set, demonstrate the effectiveness of the built custom HCQNN, using the better features of the previous tutorials, while also testing different encoding methods and optimizers. These results were better than the direct comparison with the first tutorial.

RUNNING CUSTOM HCQNN ON REAL HARDWARE

CONCLUSION

In this report, i explored the implementation of hybrid quantum convolutional neural networks (HCQNNs) using Qiskit and PyTorch. We began by understanding the basics of quantum neural networks (QNNs) and their applications in image classification tasks. Through hands-on tutorials, we learned how to construct QNNs, integrate them with classical layers, and train them on datasets like MNIST. We also delved into advanced concepts such as feature mapping, ansatz design, and the use of different optimizers. Finally, we compared various QNN architectures and proposed a custom HCQNN that leverages the strengths of both classical and quantum computing for improved performance in image classification tasks. The results obtained from the experiments demonstrated the potential of HCQNNs in achieving competitive accuracy on image classification tasks. The combination of classical and quantum layers allows for efficient feature extraction and representation learning, making HCQNNs a promising avenue for future research in quantum machine learning.

REFERENCES

REFERENCES

[1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019, pp. 8024–8035. [Online]. Available: https://arxiv.org/abs/1912.01703

[2] Y. LeCun and C. Cortes, "MNIST handwritten digit database," http://yann.lecun.com/exdb/mnist/, 2010, accessed: 2025-05-20.

[3] PyTorch Contributors, "torch.utils.data — pytorch 2.7 documentation," https://pytorch.org/docs/stable/data.html, 2025, accessed: 2025-05-20.

[4] Qiskit Contributors, "Zfeaturemap class — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.ZFeatureMap, 2025, accessed: 2025-05-20.

[5] ——, "Zzfeaturemap class — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.ZZFeatureMap, 2025, accessed: 2025-05-20.

[6] ——, "Paulifeaturemap class — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.PauliFeatureMap, 2025, accessed: 2025-05-20.

[7] ——, "Efficientsu2 ansatz — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.EfficientSU2, 2025, accessed: 2025-05-20.

[8] ——, "realamplitudes ansatz — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.RealAmplitudes, 2025, accessed: 2025-05-20.

[9] ——, "Twolocal ansatz — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.TwoLocal, 2025, accessed: 2025-05-20.

[10] PyTorch Contributors, "Neural networks — pytorch documentation," https://docs.pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html, 2025, accessed: 2025-05-20.

[11] ——, "Dropout2d — pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout2d.html, 2025, accessed: 2025-05-20.

[12] ——, "Relu — pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.ReLU.html, 2025, accessed: 2025-05-20.

[13] ——, "Adam — pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html, 2025, accessed: 2025-05-20.

[14] Qiskit Contributors, "Sparsepauliop — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.quantum_info.SparsePauliOp, 2025, accessed: 2025-05-20.

[15] ——, "Estimatorqnn — qiskit documentation," https://qiskit-community.github.io/qiskit-machine-learning/stubs/qiskit_machine_learning.neural_networks.EstimatorQNN.html, 2025, accessed: 2025-05-20.

[16] ——, "Cobyla — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/0.37/qiskit.algorithms.optimizers.COBYLA, 2025, accessed: 2025-05-20.